M16 - DataOps, CI/CD, Security & Observability

Zero-to-Job-Ready Data Engineer 2026 | RC2 | 14 beginner-complete lessons

BEGINNER SUFFICIENCY RULE - Learn the operational reason first, then the tool/configuration. Every lesson ends with evidence you can inspect and a failure you can diagnose.

M16-L01 - Production Mindset

Why this matters

A pipeline that ran successfully once is only a demonstration. Production readiness means the result stays correct, observable, secure and recoverable when inputs, people and environments change.

Start with the simple idea

Think of a production data system as a promise: the right data, for the right consumers, at the expected time, with evidence that lets another engineer understand what happened.

Technical model

- Process success: the program or scheduler completed.
- Data success: the published data satisfies freshness, completeness, validity and reconciliation expectations.
- Change safety: code/configuration changes are reviewed, tested and traceable to a version.
- Operational safety: failures have owners, evidence, retry/backfill/rollback paths and known escalation steps.
- Access safety: identities receive only the actions and scopes they need.

Worked reasoning example

A daily order load exits with code 0 but reads yesterday's file. Scheduler health is green; business correctness is red. A production design records the source identifier, expected business date, row counts and freshness, and fails publication when those controls disagree.

Evidence to inspect

- Run ID and deployed commit/version.
- Source file/object/table identifier and business date.
- Source, valid, rejected and loaded row counts.
- Freshness and control totals.
- Owner/runbook link for the job.

Common mistakes and debugging

- Treating "green DAG" as proof of correct data.
- Keeping recovery knowledge only in the author's memory.
- Editing production directly without change evidence or rollback path.

Before practice

G01 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes
Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L02 - Testing Strategy for Data Engineering

Why this matters
One test type cannot protect every failure boundary. Data systems need a portfolio of tests that fail close to the defect and a smaller number of expensive whole-flow checks.

Start with the simple idea
Test the smallest thing that can prove a rule, then add boundary tests where systems meet. Use end-to-end tests to prove integration, not to replace every smaller test.

Technical model
- Unit test: deterministic function/transformation behavior with small fixtures.
- Integration test: file/database/API/tool boundary behavior.
- Data-contract test: schema, uniqueness, nulls, accepted values, referential integrity, freshness or reconciliation.
- End-to-end test: representative source-to-trusted-output flow.
- Regression test: a previously observed defect becomes a permanent test case.

Worked reasoning example
A deduplication function can pass unit tests while the production job still points to the wrong database. A database integration test catches connection/table behavior; a data test catches duplicate business keys; a small end-to-end test catches incorrect wiring.

Evidence to inspect
- Use tiny fixed fixtures so expected results are obvious.
- Make a test fail for the intended reason before trusting it.
- Give failures messages that name the violated rule.
- Do not mock every boundary; some real boundary behavior must be exercised.

Common mistakes and debugging
- Only happy-path tests.
- Huge tests whose failure does not reveal the broken layer.
- Checking only row count while ignoring business correctness.

Before practice
G01 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes
Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L03 - Automated Quality Gates

Why this matters

A rule is much stronger when it can automatically stop an unsafe merge, deployment or publication.

Start with the simple idea

A quality gate converts an expectation into machine-readable pass/fail evidence. Advisory warnings inform; required gates block progression.

Technical model

- Code gate: syntax, tests, static checks and packaging validation.
- Data gate: required fields, keys, values, reconciliation, freshness and volume thresholds.
- Release gate: all critical checks required before promotion.
- Exit status matters: a failing rule must return non-zero to the calling system.
- Critical floors matter: some failures cannot be averaged away by a high total score.

Worked reasoning example

A customer-deduplication change passes syntax checks but violates uniqueness. CI should fail before merge. A later production-quality gate may also block downstream publication if the source-to-target reconciliation is wrong.

Concrete pattern

```
python G02_QUALITY_GATE.py data/orders_good.csv
# exit 0 = gate passed
python G02_QUALITY_GATE.py data/orders_bad.csv
# non-zero = gate blocked
```

Evidence to inspect

- Run a good fixture and save PASS evidence.
- Run a deliberately bad fixture and prove the command fails.
- Read the exact failed rule; do not simply weaken the threshold.
- Only change the rule when the business contract changed and that change is documented.

Common mistakes and debugging

- Checks that print WARNING but exit successfully.
- continue-on-error on critical controls.
- Making thresholds so wide they never detect anything.

Before practice

G02 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L04 - Continuous Integration Fundamentals

Why this matters

CI gives every proposed code change the same reproducible verification instead of relying on a developer remembering a checklist.

Start with the simple idea

Repository event -> workflow -> job -> steps -> status. The runner starts clean, so undeclared laptop dependencies are exposed early.

Technical model

- Trigger intentionally on pull_request and/or selected pushes.
- Install the runtime and dependencies explicitly.
- Run the same core commands locally and in CI where possible.
- Keep checks fast enough for regular use; move slow exhaustive tests to an appropriate later stage.
- Store useful logs/test reports as evidence when needed.

Worked reasoning example

A developer has a local package installed globally, so code works on their laptop. The clean CI runner fails at import. That failure is useful: the dependency was never declared.

Concrete pattern

```
pull_request -> clean runner -> install runtime -> run tests -> run data gate -> status
```

Evidence to inspect

- Reproduce the exact failing CI command locally.
- Compare runtime version, dependency versions, working directory and environment variables.
- Inspect the first meaningful failure rather than editing YAML randomly.

Common mistakes and debugging

- CI only after merge to main.
- A workflow that hides command failures.
- Environment-dependent scripts that assume your personal folder structure.

Before practice

G03 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L05 - GitHub Actions for Data Projects

Why this matters

GitHub Actions turns repository events into visible automated evidence and can separate verification from protected deployment.

Start with the simple idea

A workflow YAML file defines events, permissions, jobs, runners and steps. A job may reference an environment whose secrets/variables and protection rules differ from another environment.

Technical model

- Use least-privilege workflow permissions such as contents: read when writing is unnecessary.
- Reference secrets through the secrets context; never paste values into YAML or echo them.
- Separate CI from deployment jobs.
- Use environment protections/approvals for sensitive deployment when available.
- Pin or deliberately choose action versions and understand what third-party actions can access.

Worked reasoning example

A PR job checks out code, sets up Python, installs dependencies and runs tests. A separate production deployment job references a protected prod environment and runs only after required checks/approval.

Concrete pattern

```
permissions:
  contents: read
...
- run: python -m unittest discover -s tests -v
- run: python G02_QUALITY_GATE.py data/orders_good.csv
```

Evidence to inspect

- Open the failed job and identify the exact failed step.
- Compare its command with the local command.
- If credentials are involved, inspect whether the secret exists and is scoped correctly without printing it.

Common mistakes and debugging

- write-all permissions by default.
- Production deployment from every branch.
- Putting a token in YAML because the repository is “private”.

Before practice

G03/G04 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L06 - Dev, Test and Prod Environments

Why this matters

Safe delivery requires places where changes can be tried and verified without silently becoming production behavior.

Start with the simple idea

Keep one codebase where possible; externalize environment-specific values such as endpoints, dataset names and capacities. Secrets stay outside ordinary config.

Technical model

- Development: fast iteration, disposable data/resources.
- Test/staging: representative verification and promotion checks.
- Production: controlled access, real consumers, monitoring and recovery requirements.
- Configuration can change by environment; core business logic should not fork into three unrelated codebases.
- Promotion records which version moved to which environment.

Worked reasoning example

The same pipeline reads APP_ENV. dev points to orders_dev, test to orders_test and prod to orders_gold. The code path stays shared; credentials are supplied by the environment secret store.

Concrete pattern

```
APP_ENV=dev  -> config/dev.json
APP_ENV=test -> config/test.json
APP_ENV=prod -> config/prod.json
# credentials are not stored in these files
```

Evidence to inspect

- Print only non-sensitive resolved configuration.
- Confirm the environment value explicitly before destructive or expensive actions.
- Use environment-specific test data and access boundaries.

Common mistakes and debugging

- Copy-pasting three codebases.
- Using production credentials during local development.
- A default environment that silently points to production.

Before practice

G04 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L07 - Deployment and Rollback

Why this matters

Every deployment should answer three questions: what changed, how was it verified, and how will you recover if validation fails?

Start with the simple idea

Promotion moves an approved version to an environment. Rollback restores a known-good code/configuration state; data changes may require a forward-fix or controlled data repair instead.

Technical model

- Record commit/tag/build artifact and target environment.
- Run pre-deployment checks and capture results.
- Perform post-deployment validation against technical and data controls.
- Design and test the recovery path before an incident.
- Separate code rollback from data rollback: they are not always the same operation.

Worked reasoning example

A Gold-model release changes revenue logic. After deployment, control totals exceed the approved tolerance. The team stops downstream publication, identifies the deployed version, restores the previous code or fixes forward, then reprocesses only the affected period and validates totals again.

Evidence to inspect

- Preserve the failing version and validation output.
- Know which historical data was produced by the bad version.
- Validate after rollback/rerun; “command succeeded” is not enough.

Common mistakes and debugging

- No record of deployed commit.
- Rollback script never tested.
- Assuming code rollback automatically repairs already-published bad data.

Before practice

Independent P04 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L08 - Secrets and Credential Safety

Why this matters

A leaked credential can convert an ordinary coding mistake into unauthorized data access.

Start with the simple idea

Secrets include passwords, tokens, keys and connection credentials. Keep them out of source code, commit history, screenshots, logs and normal configuration.

Technical model

- Use repository/platform/environment secret stores when appropriate.
- Prefer short-lived workload identity over long-lived personal tokens when the platform supports it.
- Mask logs and avoid exposing values on command lines.

- If a secret may have leaked: stop use, revoke/rotate, remove exposure, inspect access/history and add prevention.
- .gitignore prevents future tracking; it does not erase a value already committed.

Worked reasoning example

A workflow references a secret name. The job receives the value at runtime, but logs never print it. If the token is accidentally committed, deleting the line in the next commit is insufficient; the credential must be revoked or rotated.

Concrete pattern

```
DATA_TOKEN = runtime secret supplied by platform
application reads DATA_TOKEN
application never prints DATA_TOKEN
```

Evidence to inspect

- Search code/config/logs for secret-like assignments without printing values.
- Treat suspicious exposure as an incident, not merely a formatting problem.
- Verify the old credential no longer works after rotation when appropriate.

Common mistakes and debugging

- Base64 treated as encryption.
- .env ignored after a token was already committed.
- Logging request headers containing authorization values.

Before practice

G05 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L09 - RBAC and Least Privilege

Why this matters

Permissions should match the job being performed, not the easiest role that removes an error.

Start with the simple idea

RBAC maps identities to roles/actions at a scope. Least privilege means the minimum permissions needed, at the smallest practical scope, for the necessary duration.

Technical model

- Separate human identities from unattended service/workload identities.
- Distinguish read, write, share and administration actions.
- Scope access to workspace/item/data boundaries.
- Temporary elevated access should expire or be removed.
- Access should be reviewed when owners or responsibilities change.

Worked reasoning example

An analyst needs read access to curated Gold data. A pipeline identity needs read raw + write Silver/Gold. Neither automatically needs workspace admin, tenant admin or permission to share data.

Evidence to inspect

- When you see 403/permission denied, first identify the exact required action and scope.
- Test required actions after reducing permissions.
- Document why each permission exists.

Common mistakes and debugging

- Admin as the universal fix.
- Shared personal account for automation.
- Broad roles that are never reviewed.

Before practice

G06 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L10 - PII, Encryption and Masking

Why this matters

Data engineers move sensitive information even when they are not security specialists. The pipeline should minimize unnecessary exposure.

Start with the simple idea

Classify sensitive fields first. Encryption protects data in transit/at rest; masking or tokenization changes what a consumer sees. Data minimization asks whether the raw field is needed at all.

Technical model

- Classify direct identifiers and sensitive attributes before publication.
- Do not replicate sensitive columns into every layer by default.
- Use masking/pseudonymization/tokenization when consumers need linkage but not the raw value.
- Use encryption for storage/transport protection; manage encryption keys separately.
- Keep sensitive values out of debug logs and test fixtures where possible.

Worked reasoning example

A Gold customer mart needs region and segment, not raw email or phone. The raw restricted layer keeps those fields; the published mart omits them or emits a stable approved token.

Evidence to inspect

- Inspect the final published schema, not only the UI display.
- Verify exports/logs do not reintroduce raw values.
- Document the business reason for retaining any sensitive field.

Common mistakes and debugging

- Hashing treated as universal anonymization.
- Masking a screen while exporting the raw value.
- Using real production PII in beginner test data.

Before practice

G07 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L11 - Metadata, Lineage and Governance

Why this matters

When trusted data is wrong, the team needs to know what it means, where it came from, who owns it and what will break if it changes.

Start with the simple idea

Metadata describes datasets and jobs. Lineage records dependencies/movement. Governance assigns ownership, classifications, policies and safe-use expectations.

Technical model

- Technical metadata: schema, types, partitions, timestamps, job identifiers.
- Business metadata: meaning, grain, owner, SLA and classification.
- Operational metadata: last successful run, freshness and quality status.
- Lineage: source -> transformations -> target -> consumer relationships.
- Governance is useful when it connects to real access, quality and ownership decisions.

Worked reasoning example

A Silver field is renamed. Lineage reveals three Gold tables and one dashboard downstream. The engineer can sequence a compatible change and warn the owners instead of discovering impact after deployment.

Evidence to inspect

- Trace upstream and downstream before changing a critical field.
- Ensure critical datasets have owner + grain.
- Keep lineage close to code/platform so it can be maintained.

Common mistakes and debugging

- No owner for business-critical data.
- A lineage diagram that is never updated.
- Governance documentation disconnected from actual permissions and quality checks.

Before practice

G08 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes
Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L12 - Observability for Data Pipelines

Why this matters
You cannot reliably operate a data system when you cannot infer whether it is healthy and whether its outputs are trustworthy.

Start with the simple idea
Observability combines runtime and data signals. Logs explain events, metrics quantify behavior, freshness/volume/quality describe data state, and alerts connect abnormal signals to a response.

Technical model
- Logs: run identifiers, stages, errors and important decisions.
- Runtime metrics: duration, retries, resource indicators.
- Data metrics: source/loaded/rejected rows, control totals, freshness and quality scores.
- Downstream signal: whether expected consumers/tables were successfully refreshed.
- Alerts need thresholds, owner and action; otherwise they become noise.

Worked reasoning example
A pipeline takes 95 minutes instead of 12 while source volume is unchanged and rejects jump from 1 to 28. The combination suggests a data/input issue more strongly than a simple need for more compute.

Evidence to inspect
- Compare current metrics with a known normal baseline.
- Look for correlated changes: volume, rejects, file counts, freshness and duration.
- Preserve run metrics before restarting.

Common mistakes and debugging
- Alerting on every warning.
- Only CPU/memory monitoring with no data health.
- Dashboards with no owner or response thresholds.

Before practice
G09 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes
Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L13 - Incidents, Runbooks and Root Cause

Why this matters
Production failures need a repeatable response that protects evidence and good data while restoring service.

Start with the simple idea

A useful incident flow is detect -> triage -> contain -> recover -> validate -> learn. A runbook documents verified actions; root-cause analysis explains technical and control contributors instead of blaming a person.

Technical model

- Detect: capture alert/run id/time and initial impact.
- Triage: classify source, data, code, configuration, permissions or capacity.
- Contain: stop unsafe downstream effects without destroying evidence.
- Recover: use known rerun/backfill/rollback/forward-fix path.
- Validate: reconcile data and confirm downstream freshness.
- Learn: document causes, missing controls, owners and deadlines for prevention work.

Worked reasoning example

A source schema change breaks a daily load. The engineer saves the failing schema/logs, pauses downstream publication, adds a compatible transformation, backfills the missed date, validates totals/freshness and records a schema-contract prevention action.

Evidence to inspect

- Write a timeline before memory fades.
- Separate trigger from deeper contributing conditions.
- Validate recovery with data controls, not only process status.

Common mistakes and debugging

- Deleting/recreating before preserving evidence.
- Root cause = “engineer made a mistake”.
- No post-recovery validation.

Before practice

G09 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.

M16-L14 - Terraform and IaC Foundations

Why this matters

Infrastructure changes are easier to review, repeat and audit when desired configuration is expressed as code.

Start with the simple idea

Terraform configuration describes desired resources. init prepares the working directory, plan compares configuration/state/real objects and previews changes, and apply executes an approved change. State records Terraform’s mapping to managed objects.

Technical model

- HCL files define resources, data, variables, locals and outputs.

- Providers connect Terraform to external APIs; this course also uses terraform_data so basic validation can work without a cloud account.
- Variables make configuration reusable; do not place secret values in committed tfvars.
- State can contain sensitive operational information and should be protected.
- Review plans before apply, especially destroys/replacements and unexpected drift.
- Use fmt/validate before plan; avoid auto-approve as a default production habit.

Worked reasoning example

A production plan says 1 create, 2 updates and 1 destroy. The correct response is not “the plan succeeded, so apply it.” Review which object will be destroyed, why state/config disagree, whether the target environment is correct and whether the change is authorized.

Concrete pattern

```
terraform fmt -check
terraform validate
terraform plan -var="environment=dev"
# review before any apply
```

Evidence to inspect

- Run terraform fmt -check and validate when Terraform is installed.
- Inspect the full plan, not just the summary counts.
- Stop when there is unexplained destroy/replace behavior.
- Never commit state or secret tfvars to a public repository.

Common mistakes and debugging

- Blind apply -auto-approve.
- Manual cloud changes with ignored drift.
- Editing state casually to make an error disappear.

Before practice

G10 turns this lesson into observable evidence. Save the attempt before opening review material.

Explain it without notes

Explain the control in 3-5 sentences, name the evidence you would inspect, and state one failure that the control prevents or detects.